

Python

Знакомство с коллекциями: СПИСКИ



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

- Итерируемый объект
- Цикл for
- Функция range
- Операторы break, continue, else в цикле for
- Вложенные циклы
- Вложенные циклы с использованием while и for

План занятия

- Коллекции
- Индексация списков
- Изменение по индексу
- Операции со списками
- Цикл for со списками



ОСНОВНОЙ БЛОК





Коллекции



Коллекции

Это структуры данных, которые позволяют хранить и организовывать множество элементов.



Изменяемые типы данных

Это такие структуры данных, которые позволяют изменять своё содержимое после создания объекта.



Список (list)

Это упорядоченная изменяемая (mutable) коллекция, которая может содержать элементы любых типов.

Основные характеристики списка



Изменяемость: Элементы списка можно изменять после его создания.



Упорядоченность: Элементы списка хранятся в том порядке, в котором они были добавлены.



Поддержка дубликатов: Список может содержать повторяющиеся элементы.



Поддержка различных типов данных: Список может содержать элементы разных типов.



Индексация: К элементам списка можно обращаться по индексу.

Пример создания списка



```
fruits = ["apple", "banana", "cherry"] # Список со строками
```

```
print(fruits)
```

```
numbers = [1, 2, 3] # Список с числами
```

```
print(numbers)
```

Хранение разных типов элементов в списке



Определение

Список (list) является универсальной структурой данных, которая может хранить элементы разных типов данных.

Код

```
mixed_list = [42, "Python", 3.14, True, [1, 2, 3]]  
  
print(mixed_list)
```

Создание списка с элементами



Определение

Список создаётся с использованием квадратных скобок [], а элементы внутри разделяются запятыми.

Код

```
my_list = [1, 2, 3, 4]  
  
print(my_list)
```

Создание пустого списка



Квадратные скобки

```
empty_list = []  
  
print(empty_list)
```

Функция list()

```
my_list = list()  
  
print(my_list)
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Что можно хранить в списке в Python?

- a. Только строки
- b. Только числа
- c. Только числа и строки
- d. Элементы разных типов данных



Выберите правильный вариант ответа

Что можно хранить в списке в Python?

- a. Только строки
- b. Только числа
- c. Только числа и строки
- d. Элементы разных типов данных**



Выберите правильные варианты ответа

Какой результат выведет следующий код?

```
my_list = list()
```

```
print(my_list)
```

- a. []
- b. [None]
- c. list()
- d. Ошибка



Выберите правильные варианты ответа

Какой результат выведет следующий код?

```
my_list = list()
print(my_list)
```

- a. []
- b. [None]
- c. list()
- d. Ошибка



ВОПРОСЫ





Индексация и срезы СПИСКОВ



Индексация списков

Это процесс доступа к элементам списка по их позиции (индексу) внутри списка.

Пример индексации



```
my_list = ["apple", "banana", "cherry", "date"]
```

```
# Доступ к первому элементу
```

```
print(my_list[0])
```

```
# Доступ к последнему элементу с положительным индексом
```

```
print(my_list[3])
```

```
# Доступ к последнему элементу с отрицательным индексом
```

```
print(my_list[-1])
```

```
# Доступ к первому элементу с отрицательным индексом
```

```
print(my_list[-4])
```



Срезы списков (slices)

Это способ получения части списка (подсписка), без изменения исходного списка. Срезы позволяют выбрать элементы по индексу с указанием начала, конца и шага.

Основной синтаксис срезов



start (необязательно) — индекс начала среза (включительно). По умолчанию — 0.



stop (необязательно) — индекс конца среза (не включительно).



step (необязательно) — шаг, с которым нужно выбирать элементы. По умолчанию — 1.

Срезы положительным индексом



```
my_list = [0, 1, 2, 3, 4, 5, 6]
# Срез с 1-го по 4-й элемент (не включительно)
print(my_list[1:4])
# Срез каждого второго элемента
print(my_list[::2])
# Срез каждого второго элемента в обратном порядке
print(my_list[::-2])
# Срез от 4-го элемента до начала
print(my_list[4::-1])
# Срез с 3-го элемента до конца
print(my_list[3:])
# Копия списка с помощью среза
print(my_list[:])
```

Срезы отрицательным индексом



```
my_list = [0, 1, 2, 3, 4, 5, 6]

# Срез от -4-го до -1-го элемента
print(my_list[-4:-1])

# Срез от -2-го до -5-го элемента
print(my_list[-2:-5:-1])

# Срез списка в обратном порядке
print(my_list[::-1])
```



ВОПРОСЫ





Изменение по индексу



Изменение по индексу

В Python элементы списка можно изменять напрямую, используя индексацию.

Пример изменения элементов



```
my_list = ["apple", "banana", "cherry"]
```

```
# Изменим второй элемент (с индексом 1)
```

```
my_list[1] = "blueberry"
```

```
print(my_list)
```

Изменение по срезу



```
my_list = [10, 20, 30, 40, 50]
```

```
# Заменяем второй и третий элементы
```

```
my_list[1:3] = [200, 300]
```

```
print(my_list)
```

Использование отрицательных индексов для изменения



```
my_list = [100, 200, 300, 400]
```

```
# Заменяем последний элемент
```

```
my_list[-1] = 500
```

```
print(my_list)
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [1, 2, 3, 4]
```

```
print(my_list[2])
```

- a. 1
- b. 2
- c. 3
- d. 4



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [1, 2, 3, 4]
```

```
print(my_list[2])
```

- a. 1
- b. 2
- c. 3**
- d. 4



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [0, 1, 2, 3, 4, 5]
```

```
print(my_list[::-1])
```

- a. [0, 1, 2, 3, 4, 5]
- b. [5, 4, 3, 2, 1, 0]
- c. [0, -1, -2, -3, -4, -5]
- d. Ошибка



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [0, 1, 2, 3, 4, 5]
```

```
print(my_list[::-1])
```

- a. [0, 1, 2, 3, 4, 5]
- b. [5, 4, 3, 2, 1, 0]**
- c. [0, -1, -2, -3, -4, -5]
- d. Ошибка



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = ["a", "b", "c", "d"]
```

```
my_list[1] = 0
```

```
print(my_list)
```

- a. ['a', 'b', 'c', 'd']
- b. ['a', 0, 'c', 'd']
- c. ['0', 'b', 'c', 'd']
- d. ['a', 0, 'c', 'd']



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = ["a", "b", "c", "d"]
```

```
my_list[1] = 0
```

```
print(my_list)
```

- a. ['a', 'b', 'c', 'd']
- b. ['a', 0, 'c', 'd']**
- c. ['0', 'b', 'c', 'd']
- d. ['a', 0, 'c', 'd']



ВОПРОСЫ





Операции со списками



Операции со списками

Списки в Python поддерживают множество операций, позволяющих изменять, объединять или проверять списки. Ниже приведены основные операции со списками.

Конкатенация списков (объединение)



```
list1 = [1, 2, 3]

list2 = [4, 5, 6]

combined_list = list1 + list2

print(combined_list)
```

Повторение списка



```
repeated_list = [0] * 5

print(repeated_list)

my_list = [1, 2, 3]

repeated_list = my_list * 3

print(repeated_list)
```

Проверка на наличие элемента в списке



```
my_list = [1, 2, 3, 4, 5]

print(3 in my_list)
print(6 in my_list)

my_list = ["apple", "banana", "cherry"]

print("apple" in my_list)
print("app" in my_list)  # Ищет полное совпадение элемента
```

Длина списка



```
my_list = [1, 2, 3, 4, 5]
```

```
print(len(my_list))
```

Полезно знать



Из строки

```
word = "python"
# Строка "разбивается" на отдельные символы
my_list = list(word)
print(my_list)
print(type(my_list))
```

Из объекта range:

```
my_range = range(1, 11, 2)
print(my_range)
print(type(my_range))
my_list = list(my_range)
print(my_list)
```



Сравнение списков

Списки можно сравнивать друг с другом с помощью операторов сравнения.

Примеры



```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = [1, 3, 0]

# Сравнение на равенство
print(list1 == list2)
print(list1 == list3)

# Сравнение на неравенство
print(list1 != list2)
print(list1 != list3)

### Сравнение на больше/меньше, больше или равно/меньше или равно
print(list1 < list2)
print(list3 > list1)
print(list1 <= list2)
print(list1 >= list3)
```



Поддержка операции

Для корректного сравнения списков, элементы должны быть одного типа или поддерживать операцию сравнения между собой.

Пример корректного сравнения



```
list1 = [1, 2.6, 2]
```

```
list2 = [1, 2, 3]
```

```
# int и float можно сравнивать между собой
```

```
print(list1 < list2)
```

Пример ошибки типов



```
list1 = [1, 2, "apple"]
```

```
list2 = [1, 2, 3]
```

```
# Попытка сравнить числовое значение с строкой вызовет ошибку TypeError
```

```
print(list1 < list2)
```

Пример ошибки типов

```
list1 = [1, 2, "apple"]
list2 = [1, 2, 3]

print(list1 < list2)
```



TypeError

Traceback (most recent call last)

Cell In[4], line 4

```
1 list1 = [1, 2, "apple"]
2 list2 = [1, 2, 3]
----> 4 print(list1 < list2)
```

TypeError: '<' not supported between instances of 'str' and 'int'



Особенности сравнения

Важно учитывать, что при первом нахождении отличающихся элементов интерпретатор остановит сравнение. Поэтому если после них будут находиться несравнимые типы это не приведет к ошибке.

Пример



```
list1 = [1, 2, "apple"]
```

```
list2 = [1, 3, 10]
```

```
print(list1 < list2) # Вернёт True, так как 2 < 3, сравнение остановится до строки "apple"
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined_list = list1 + list2
print(combined_list)
```

- a. [1, 2, 3, 4, 5, 6]
- b. [4, 5, 6, 1, 2, 3]
- c. [1, 2, 3], [4, 5, 6]
- d. Ошибка



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined_list = list1 + list2
print(combined_list)
```

- a. **[1, 2, 3, 4, 5, 6]**
- b. [4, 5, 6, 1, 2, 3]
- c. [1, 2, 3], [4, 5, 6]
- d. Ошибка



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [0] * 4
```

```
print(my_list)
```

- a. [4]
- b. [1, 1, 1, 1]
- c. [0]
- d. [0, 0, 0, 0]



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [0] * 4
```

```
print(my_list)
```

- a. [4]
- b. [1, 1, 1, 1]
- c. [0]
- d. [0, 0, 0, 0]



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [1, 2, 3, 4, 5]
```

```
print(6 in my_list)
```

- a. True
- b. False
- c. Ошибка
- d. Ничего не выведется



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_list = [1, 2, 3, 4, 5]
```

```
print(6 in my_list)
```

- a. True
- b. False**
- c. Ошибка
- d. Ничего не выведется



ВОПРОСЫ





Цикл for со списками

Пример



```
my_list = [1, 2, 3, 4, 5]
```

```
for item in my_list:
```

```
    print(item)
```



Цикл for со списками

Вложенные циклы for позволяют перебирать элементы списка, где каждый элемент сам по себе является итерируемым объектом, например, строкой. Внутренний цикл проходит по символам каждой строки.

Вложенный цикл for по списку строк



```
my_strings = ["apple", "banana", "cherry"]

for word in my_strings:
    for letter in word:
        print("letter:", letter)

print()
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_strings = ["cat", "dog"]
for word in my_strings:
    for letter in word:
        print(letter, end="")
    print(end=" ")
```

- a. c a t d o g
- b. catdog
- c. c a t d o g
- d. cat dog



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
my_strings = ["cat", "dog"]
for word in my_strings:
    for letter in word:
        print(letter, end="")
    print(end=" ")
```

- a. cat dog
- b. catdog
- c. cat dog
- d. cat dog**



ПРАКТИЧЕСКАЯ РАБОТА



1. Список чисел с шагом



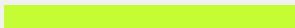
Напишите программу, которая выводит список всех чисел от 0 до n (введённого пользователем) с шагом 3.

Пример вывода:

Введите число: 15

[0, 3, 6, 9, 12, 15]

2. Список имен



Напишите программу, которая обрабатывает список строк, состоящий из имён. Нужно вывести только те имена, длина которых больше средней длины имен в списке.

Пример данных для обработки: `names = ["John", "Bob", "Alice", "Ann", "Mark"]`

Пример вывода:

Список имён: `['John', 'Bob', 'Alice', 'Anna', 'Mark']`

Имена длинее средней длины: `Alice`



ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

1. Создание списка с пользовательскими данными

Напишите программу, которая принимает список целых чисел от пользователя и добавляет каждое число в динамический массив. Программа должна принимать новое число от пользователя и добавлять его в динамический массив до тех пор, пока пользователь не введет команду "Выход". Выведите итоговый динамический массив на экран с помощью команды print.

Пример вывода:

```
Введите число или 'выход' :: 1
Введите число или 'выход' :: 2
Введите число или 'выход' :: выход
Итоговый динамический массив: [1, 2]
```

Домашнее задание

2. Квадрат нечетных чисел

Напишите программу, которая изменяет изначальный список и возводит в квадрат только нечётные числа.

```
numbers = [4, 9, 1, 7, 2, 5, 0, 3, 7, 1, 3]
```

Пример вывода:

Изначальный список: [4, 9, 1, 7, 2, 5, 0, 3, 7, 1, 3]

Измененный список: [4, 81, 1, 49, 2, 25, 0, 9, 49, 1, 9]

Заключение

